

```
# YouTube Trending Videos Cleaning Data
# Project Objective
```

```
# The objective of this project is to clean and preprocess the YouTube Trend
# identifying and handling missing values, removing duplicate records, stand
# ensuring data integrity, and detecting outliers. The cleaned dataset will
# visualization, and machine learning applications.
```

```
# import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', None)
```

```
from google.colab import files
upload = files.upload()
```

USvideos.csv
USvideos.csv(text/csv) - 62756152 bytes, last modified: 11/07/2026 - 100% done
Saving USvideos.csv to USvideos (1).csv

```
df= pd.read_csv('USvideos.csv')
original_df= df.copy() # backup copy
```

```
df.head()
```

	video_id	trending_date	title	channel_title	category_id	
0	2kyS6SvSYSE	17.14.11	WE WANT TO TALK ABOUT OUR MARRIAGE	CaseyNeistat	22	13T1
1	1ZAPwfrtAFY	17.14.11	The Trump Presidency: Last Week Tonight with J...	LastWeekTonight	24	13T0
2	5qpjK5DgCt4	17.14.11	Racist Superman Rudy Mancuso, King Bach & Le...	Rudy Mancuso	23	12T1
3	puqaWrEC7tY	17.14.11	Nickelback Lyrics: Real or Fake?	Good Mythical Morning	24	13T1
4	d380meD0W0M	17.14.11	I Dare You: GOING BALD!?	nigahiga	24	12T1

Next steps:

[Generate code with df](#)[New interactive sheet](#)

df.info()

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40949 entries, 0 to 40948
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   video_id                             40949 non-null  object
1   trending_date                         40949 non-null  object
2   title                                40949 non-null  object
3   channel_title                         40949 non-null  object
4   category_id                           40949 non-null  int64
5   publish_time                          40949 non-null  object
6   tags                                  40949 non-null  object
7   views                                 40949 non-null  int64
8   likes                                 40949 non-null  int64
9   dislikes                              40949 non-null  int64
10  comment_count                         40949 non-null  int64
11  thumbnail_link                        40949 non-null  object
12  comments_disabled                     40949 non-null  bool
13  ratings_disabled                      40949 non-null  bool
14  video_error_or_removed                40949 non-null  bool
15  description                           40379 non-null  object
dtypes: bool(3), int64(5), object(8)
memory usage: 4.2+ MB

```

```
df.describe()
```

	category_id	views	likes	dislikes	comment_count
count	40949.000000	4.094900e+04	4.094900e+04	4.094900e+04	4.094900e+04
mean	19.972429	2.360785e+06	7.426670e+04	3.711401e+03	8.446804e+03
std	7.568327	7.394114e+06	2.288853e+05	2.902971e+04	3.743049e+04
min	1.000000	5.490000e+02	0.000000e+00	0.000000e+00	0.000000e+00
25%	17.000000	2.423290e+05	5.424000e+03	2.020000e+02	6.140000e+02
50%	24.000000	6.818610e+05	1.809100e+04	6.310000e+02	1.856000e+03
75%	25.000000	1.823157e+06	5.541700e+04	1.938000e+03	5.755000e+03
max	43.000000	2.252119e+08	5.613827e+06	1.674420e+06	1.361580e+06

```
df.columns
```

```
Index(['video_id', 'trending_date', 'title', 'channel_title', 'category_id',  
      'publish_time', 'tags', 'views', 'likes', 'dislikes',  
      'comment_count',  
      'thumbnail_link', 'comments_disabled', 'ratings_disabled',  
      'video_error_or_removed', 'description'],  
      dtype='object')
```

```
# 1) Data Integrity Checks -> Data integrity ensures the dataset is accurate  
df.dtypes
```

	0
video_id	object
trending_date	object
title	object
channel_title	object
category_id	int64
publish_time	object
tags	object
views	int64
likes	int64
dislikes	int64
comment_count	int64
thumbnail_link	object
comments_disabled	bool
ratings_disabled	bool
video_error_or_removed	bool
description	object

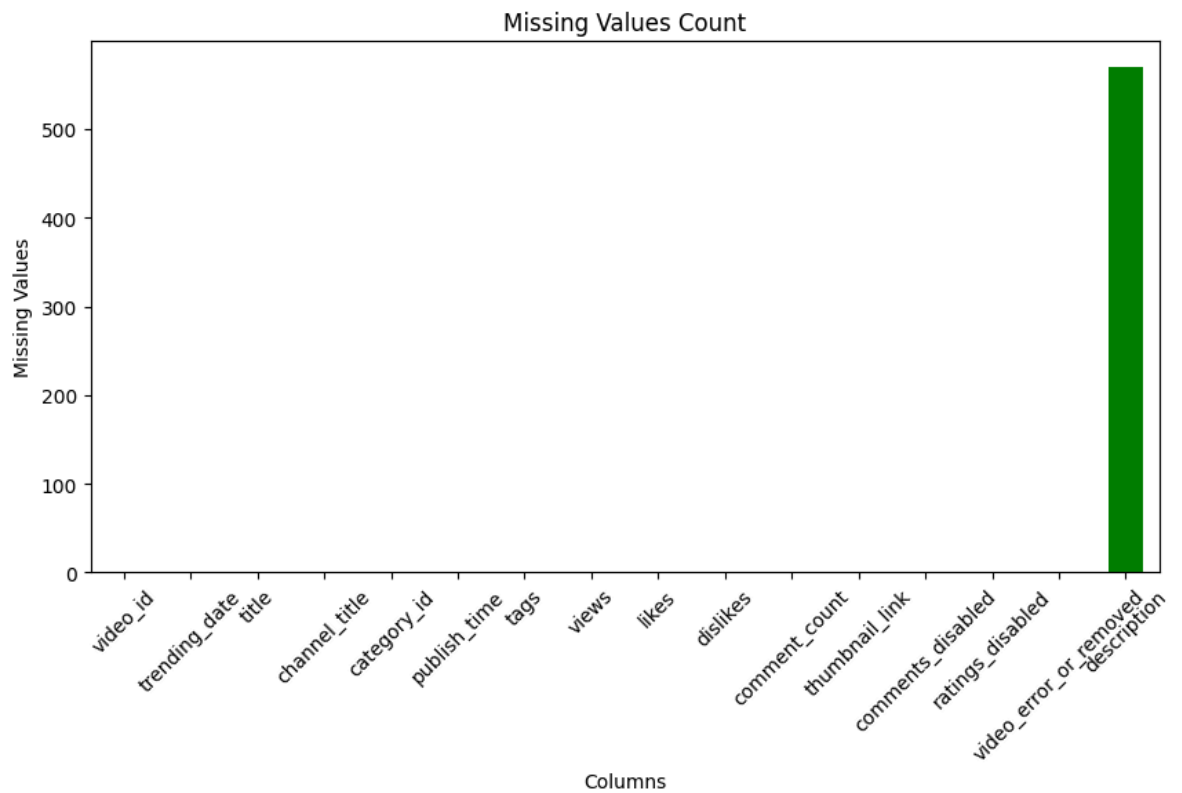
dtype: object

```
df.isnull().sum()
```

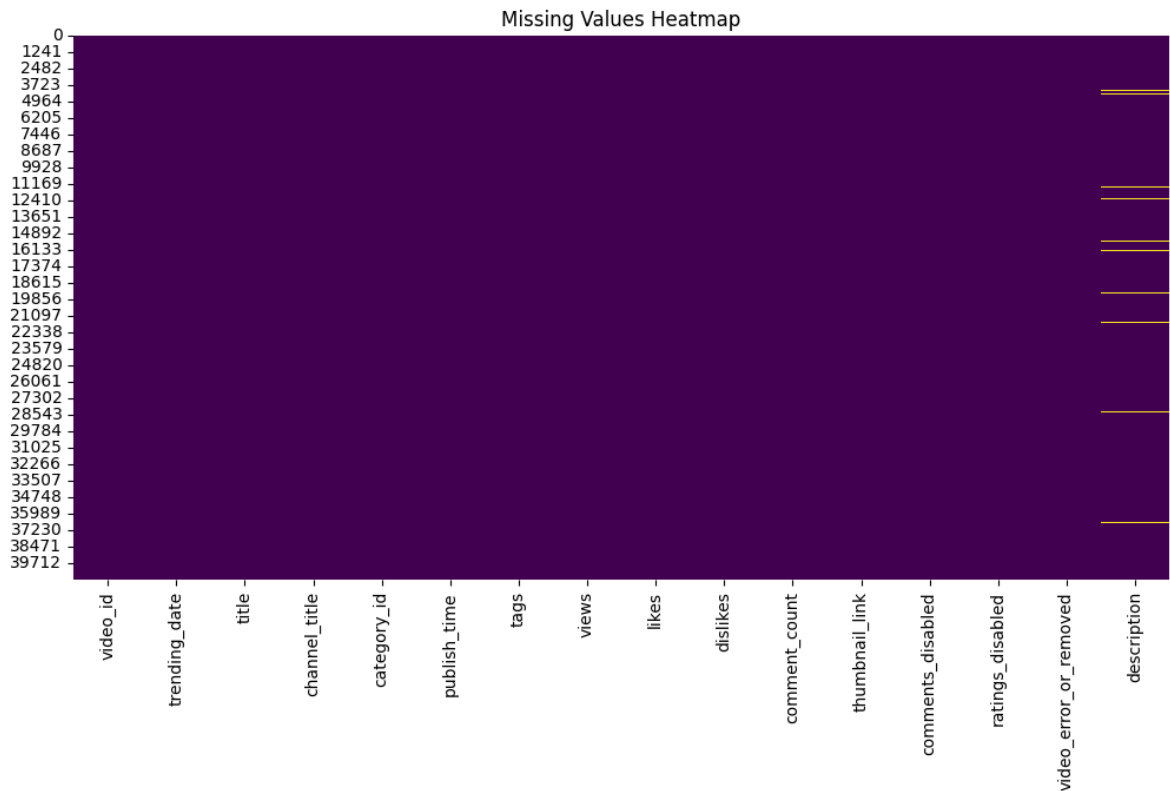
	0
video_id	0
trending_date	0
title	0
channel_title	0
category_id	0
publish_time	0
tags	0
views	0
likes	0
dislikes	0
comment_count	0
thumbnail_link	0
comments_disabled	0
ratings_disabled	0
video_error_or_removed	0
description	570

dtype: int64

```
plt.figure(figsize=(10,5))
df.isnull().sum().plot(kind='bar', color='green')
plt.title("Missing Values Count")
plt.xlabel("Columns")
plt.ylabel("Missing Values")
plt.xticks(rotation=45)
plt.show()
```



```
plt.figure(figsize=(12,6))
sns.heatmap(df.isnull(), cbar=False, cmap="viridis")
plt.title("Missing Values Heatmap")
plt.show()
```



```
duplicates = df.duplicated().sum()
duplicates
```

```
np.int64(48)
```

```
df.drop_duplicates(inplace = True)
```

```
# Check Invalid Values
(df['views'] < 0).sum()
```

```
np.int64(0)
```

```
(df['likes'] < 0).sum()
```

```
np.int64(0)
```

```
(df['dislikes'] < 0).sum()
```

```
np.int64(0)
```

```
(df['comment_count'] < 0).sum()
```

```
np.int64(0)
```

```
# check duplicate video ids
df['video_id'].duplicated().sum()

# Why?
# A video can trend on multiple days. ->So duplicate video IDs are expected,

np.int64(34550)
```

```
# Consistency Check
df['trending_date'].unique()

array(['17.14.11', '17.15.11', '17.16.11', '17.17.11', '17.18.11',
       '17.19.11', '17.20.11', '17.21.11', '17.22.11', '17.23.11',
       '17.24.11', '17.25.11', '17.26.11', '17.27.11', '17.28.11',
       '17.29.11', '17.30.11', '17.01.12', '17.02.12', '17.03.12',
       '17.04.12', '17.05.12', '17.06.12', '17.07.12', '17.08.12',
       '17.09.12', '17.10.12', '17.11.12', '17.12.12', '17.13.12',
       '17.14.12', '17.15.12', '17.16.12', '17.17.12', '17.18.12',
       '17.19.12', '17.20.12', '17.21.12', '17.22.12', '17.23.12',
       '17.24.12', '17.25.12', '17.26.12', '17.27.12', '17.28.12',
       '17.29.12', '17.30.12', '17.31.12', '18.01.01', '18.02.01',
       '18.03.01', '18.04.01', '18.05.01', '18.06.01', '18.07.01',
       '18.08.01', '18.09.01', '18.12.01', '18.13.01', '18.14.01',
       '18.15.01', '18.16.01', '18.17.01', '18.18.01', '18.19.01',
       '18.20.01', '18.21.01', '18.22.01', '18.23.01', '18.24.01',
       '18.25.01', '18.26.01', '18.27.01', '18.28.01', '18.29.01',
       '18.30.01', '18.31.01', '18.01.02', '18.02.02', '18.03.02',
       '18.04.02', '18.05.02', '18.06.02', '18.07.02', '18.08.02',
       '18.09.02', '18.10.02', '18.11.02', '18.12.02', '18.13.02',
       '18.14.02', '18.15.02', '18.16.02', '18.17.02', '18.18.02',
       '18.19.02', '18.20.02', '18.21.02', '18.22.02', '18.23.02',
       '18.24.02', '18.25.02', '18.26.02', '18.27.02', '18.28.02',
       '18.01.03', '18.02.03', '18.03.03', '18.04.03', '18.05.03',
       '18.06.03', '18.07.03', '18.08.03', '18.09.03', '18.10.03',
       '18.11.03', '18.12.03', '18.13.03', '18.14.03', '18.15.03',
       '18.16.03', '18.17.03', '18.18.03', '18.19.03', '18.20.03',
       '18.21.03', '18.22.03', '18.23.03', '18.24.03', '18.25.03',
       '18.26.03', '18.27.03', '18.28.03', '18.29.03', '18.30.03',
       '18.31.03', '18.01.04', '18.02.04', '18.03.04', '18.04.04',
       '18.05.04', '18.06.04', '18.07.04', '18.14.04', '18.15.04',
       '18.16.04', '18.17.04', '18.18.04', '18.19.04', '18.20.04',
       '18.21.04', '18.22.04', '18.23.04', '18.24.04', '18.25.04',
       '18.26.04', '18.27.04', '18.28.04', '18.29.04', '18.30.04',
       '18.01.05', '18.02.05', '18.03.05', '18.04.05', '18.05.05',
       '18.06.05', '18.07.05', '18.08.05', '18.09.05', '18.10.05',
       '18.11.05', '18.12.05', '18.13.05', '18.14.05', '18.15.05',
       '18.16.05', '18.17.05', '18.18.05', '18.19.05', '18.20.05',
       '18.21.05', '18.22.05', '18.23.05', '18.24.05', '18.25.05',
       '18.26.05', '18.27.05', '18.28.05', '18.29.05', '18.30.05',
       '18.31.05', '18.01.06', '18.02.06', '18.03.06', '18.04.06',
       '18.05.06', '18.06.06', '18.07.06', '18.08.06', '18.09.06',
       '18.10.06', '18.11.06', '18.12.06', '18.13.06', '18.14.06'],
      dtype=object)
```

```
inconsistent = df[
    (df['ratings_disabled']== True) &
    (
```



```

        (df['likes'] > 0) |
        (df['dislikes'] > 0) |
        (df['comment_count'] > 0)
    )
]
print('Inconsistent Records: ', len(inconsistent))

```

Inconsistent Records: 62

```

inconsistent_comments = df[
    (df["comments_disabled"] == True) &
    (df["comment_count"] > 0)
]

print("Inconsistent Records:", len(inconsistent_comments))
# If comments are disabled, comment count should ideally be zero.

```

Inconsistent Records: 0

```

# 2)Missing Value Handling
df.isnull().sum()

```

	0
video_id	0
trending_date	0
title	0
channel_title	0
category_id	0
publish_time	0
tags	0
views	0
likes	0
dislikes	0
comment_count	0
thumbnail_link	0
comments_disabled	0
ratings_disabled	0
video_error_or_removed	0
description	569

dtype: int64

```
df["description"] = df["description"].fillna("No Description")
```

```
df.isnull().sum()
```

	0
video_id	0
trending_date	0
title	0
channel_title	0
category_id	0
publish_time	0
tags	0
views	0
likes	0
dislikes	0
comment_count	0
thumbnail_link	0
comments_disabled	0
ratings_disabled	0
video_error_or_removed	0
description	0

dtype: int64

```
# 3) Duplicate Removal  
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.drop_duplicates(inplace = True)
```

```
# 4) Data Standardization  
# remove extra spaces  
df['title'] = df['title'].str.strip()  
df["channel_title"] = df["channel_title"].str.strip()  
df["tags"] = df["tags"].str.strip()
```

```
# convert publish time  
df['publish_time'] = pd.to_datetime(df['publish_time'])
```

```
# convert Trending Date
df['trending_date'] = pd.to_datetime(
    df['trending_date'],
    format = "%y.%d.%m"
)
```

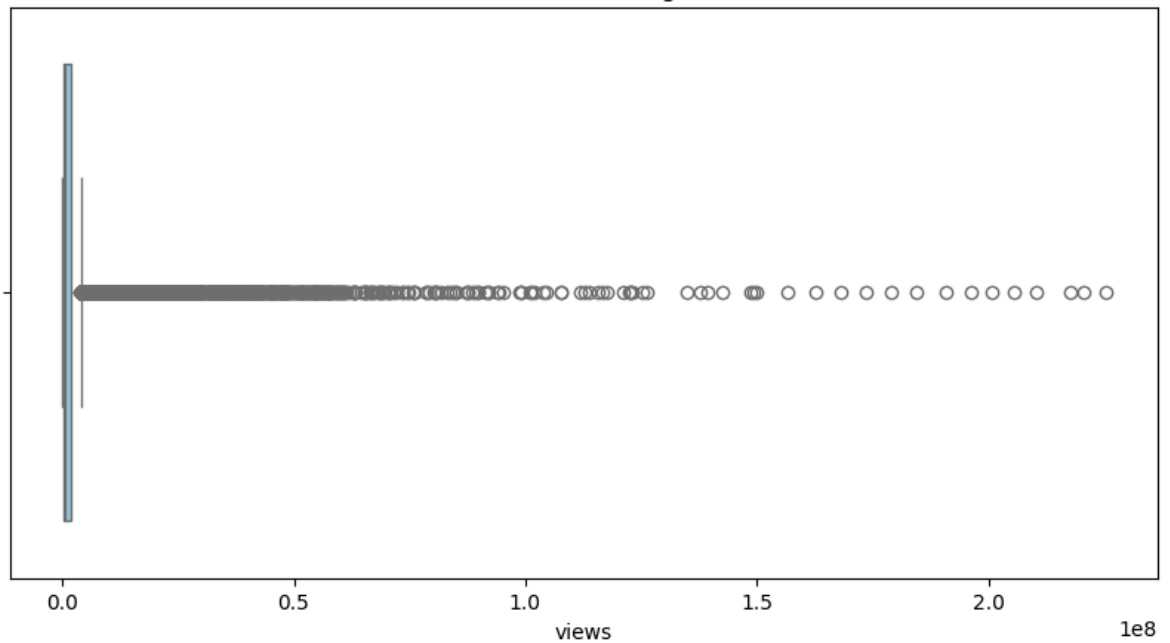
df.dtypes

	0
video_id	object
trending_date	datetime64[ns]
title	object
channel_title	object
category_id	int64
publish_time	datetime64[ns, UTC]
tags	object
views	int64
likes	int64
dislikes	int64
comment_count	int64
thumbnail_link	object
comments_disabled	bool
ratings_disabled	bool
video_error_or_removed	bool
description	object

dtype: object

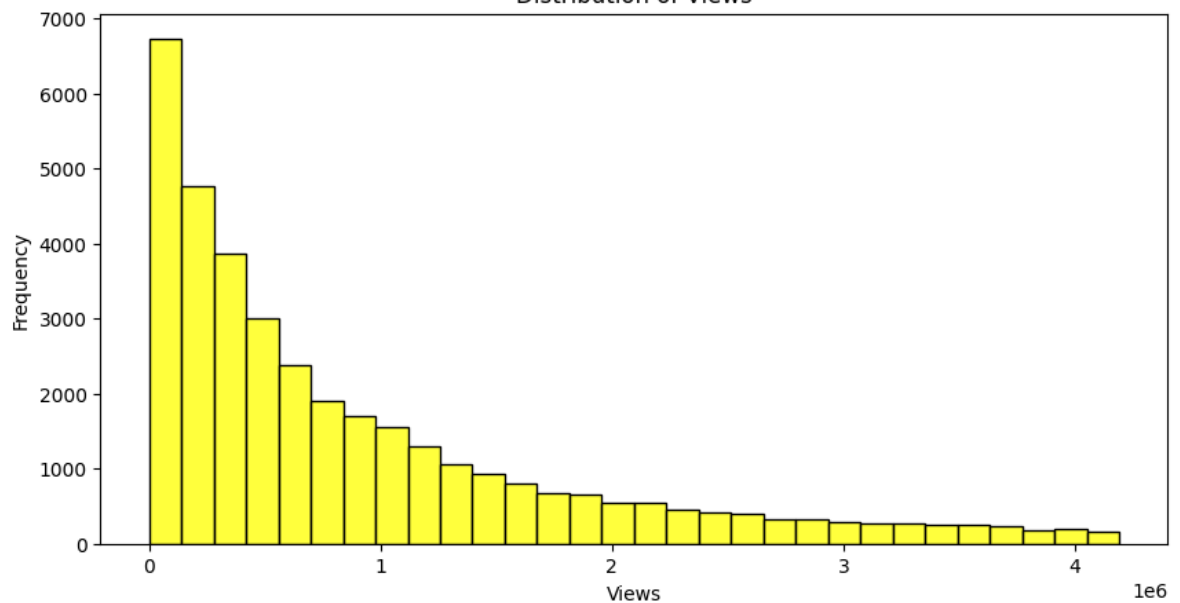
```
plt.figure(figsize=(10,5))
sns.boxplot(x=df['views'], color = 'skyblue')
plt.title("Views Before Removing Outliers")
plt.show()
```

Views Before Removing Outliers

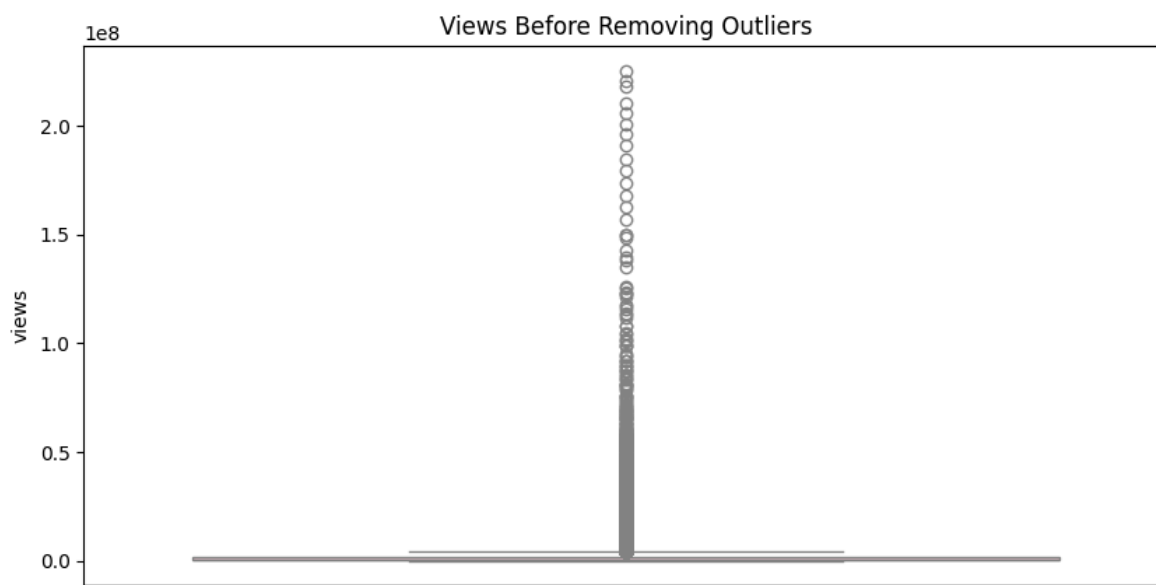


```
plt.figure(figsize=(10,5))
sns.histplot(clean_df['views'], bins=30, color='yellow')
plt.title("Distribution of Views")
plt.xlabel("Views")
plt.ylabel("Frequency")
plt.show()
```

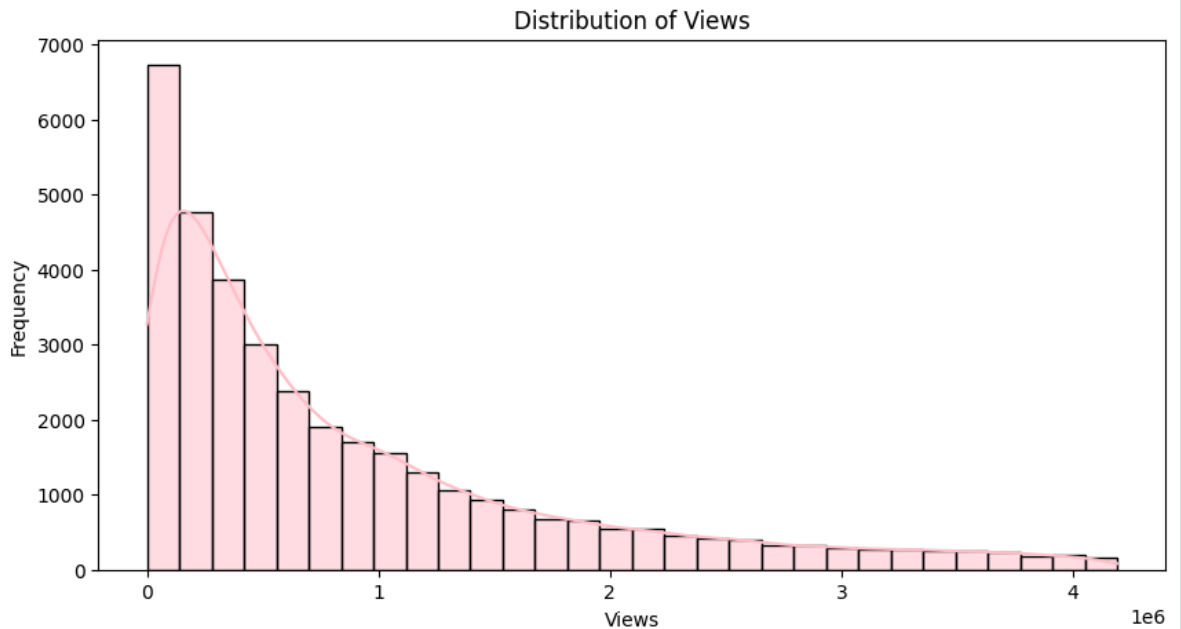
Distribution of Views



```
# Outlier Detection
plt.figure(figsize=(10,5))
sns.boxplot(df['views'], color="pink")
plt.title('Views Before Removing Outliers')
plt.show()
```



```
plt.figure(figsize=(10,5))
sns.histplot(clean_df['views'], bins=30, color = 'pink', kde= True)
plt.title("Distribution of Views")
plt.xlabel("Views")
plt.ylabel("Frequency")
plt.show()
```



```
# IQR
Q1 = df["views"].quantile(0.25)
Q3 = df["views"].quantile(0.75)

IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("Q1 =", Q1)
print("Q3 =", Q3)
print("IQR =", IQR)
print("Lower Bound:", lower)
print("Upper Bound:", upper)
```

```
Q1 = 241972.0
Q3 = 1821926.0
IQR = 1579954.0
Lower Bound: -2127959.0
Upper Bound: 4191857.0
```

```
# Detect Outliers
outliers = df[
    (df['views'] < lower) | (df['views'] > upper)
]
print("Outliers: ", len(outliers))
```

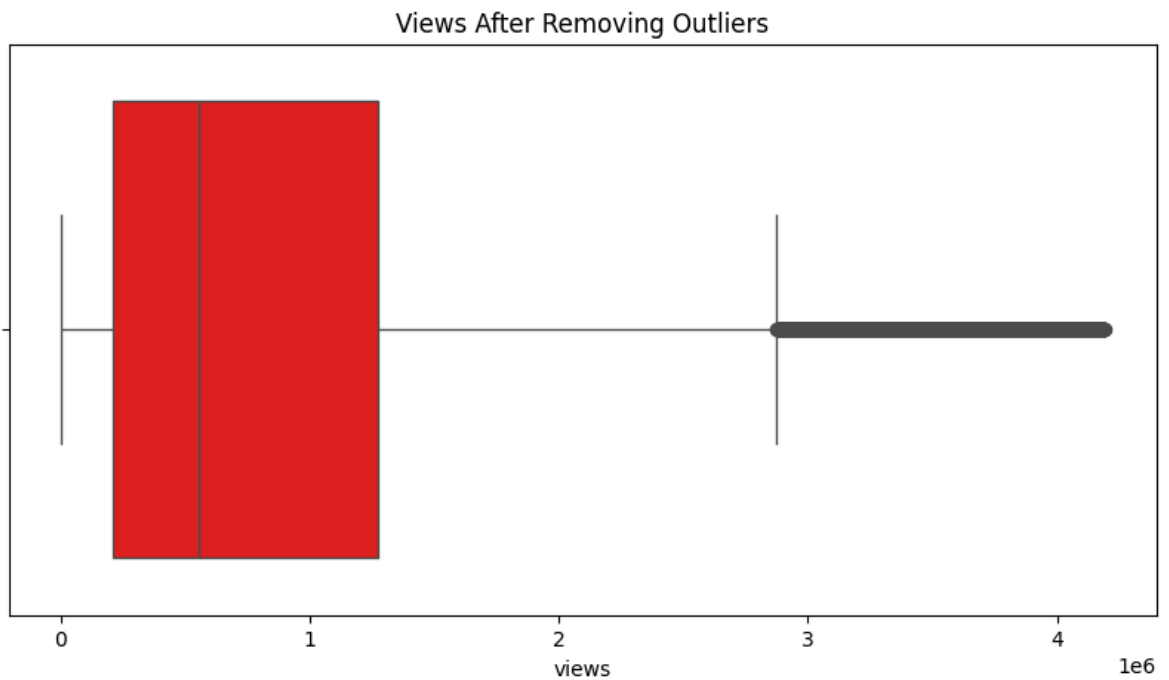
```
Outliers:  4497
```

```
# remove outliers
```

```
clean_df = df[
    (df['views']>= lower) & (df['views']<= upper)
]
print("Cleaned Dataset Shape:", clean_df.shape)
```

Cleaned Dataset Shape: (36404, 16)

```
plt.figure(figsize=(10,5))
sns.boxplot(x=clean_df["views"], color='red')
plt.title("Views After Removing Outliers")
plt.show()
```

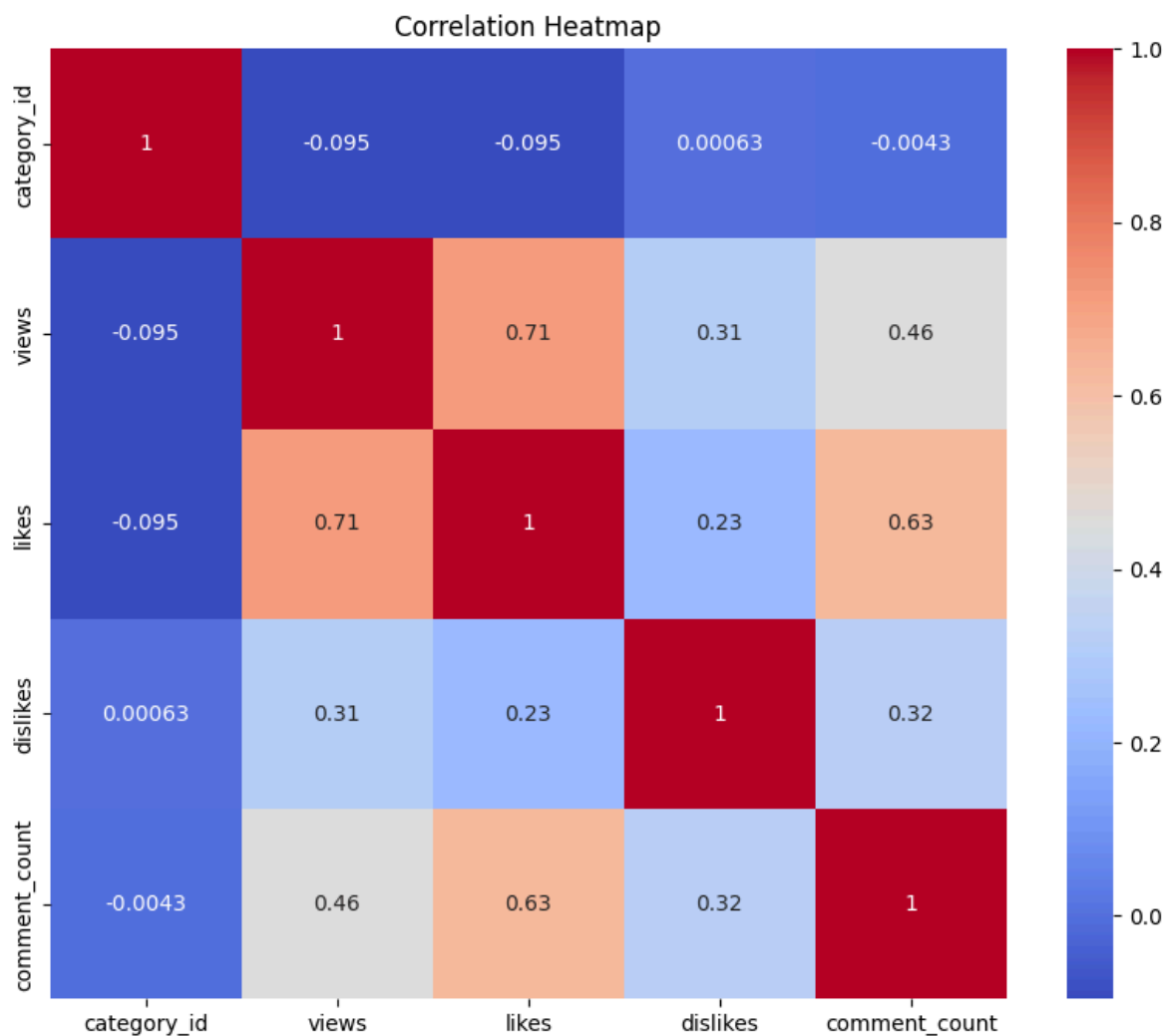


```
# final Validation
print(clean_df.isnull().sum())
```

```
video_id          0
trending_date     0
title             0
channel_title     0
category_id       0
publish_time      0
tags              0
views             0
likes             0
dislikes          0
comment_count     0
thumbnail_link    0
comments_disabled 0
ratings_disabled  0
video_error_or_removed 0
```

```
description          0
dtype: int64
```

```
numeric_df = clean_df.select_dtypes(include='number')
plt.figure(figsize=(10,8))
sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap='coolwarm'
)
plt.title("Correlation Heatmap")
plt.show()
```



```
plt.figure(figsize=(10,5))
clean_df['category_id'].value_counts().head(10).plot(kind='bar')
plt.title("Top 10 Video Categories")
plt.xlabel("Category ID")
```



```
plt.ylabel("Number of Videos")  
plt.xticks(rotation=0)
```

